INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

MONITORING AUDITING CONTROLS AND EXECUTIVE REPORTING

WEEK 5 PRACTICAL LABORATORY

NAME	**ESTHER EGBAGBE**
REGISTRATION NUMBER	**C11/26/CGRCE/17574**
COHORT	**COHORT 11**
COURSE	**GRC102**
DATE SUBMITTED	**10th of Oct, 2026**
LAB Environment	**ICDFA Kali Linux VM**

PART 1: Setting up a Simulated Environment

Task 1: Created a Docker-based environment with multiple containers



Figure 1.1: Docker ps showing the web-server, database-server and monitoring-server containers running

I created a Docker environment with three containers connected to the same network, secgov_network. The web_server runs Apache Struts 2.3.30, which is vulnerable to S2-045 (CVE-2017-5638). The database_server runs MySQL 5.7 and contains a customer_data database with fake customer records. I also set up an Ubuntu container called monitoring_server for monitoring activities later.

This set up reflects what happened in the Equifax breach, where a vulnerable, unpatched Struts application exposed a system connected to sensitive customer data. I checked the docker ps output and confirmed that all three containers are running.

The lab's *s2-045* image tag was not available on Docker Hub, so I used *vulhub/struts2:2.3.30*, which is vulnerable to the same flaw.

Task 2: Set up a vulnerability scanning system

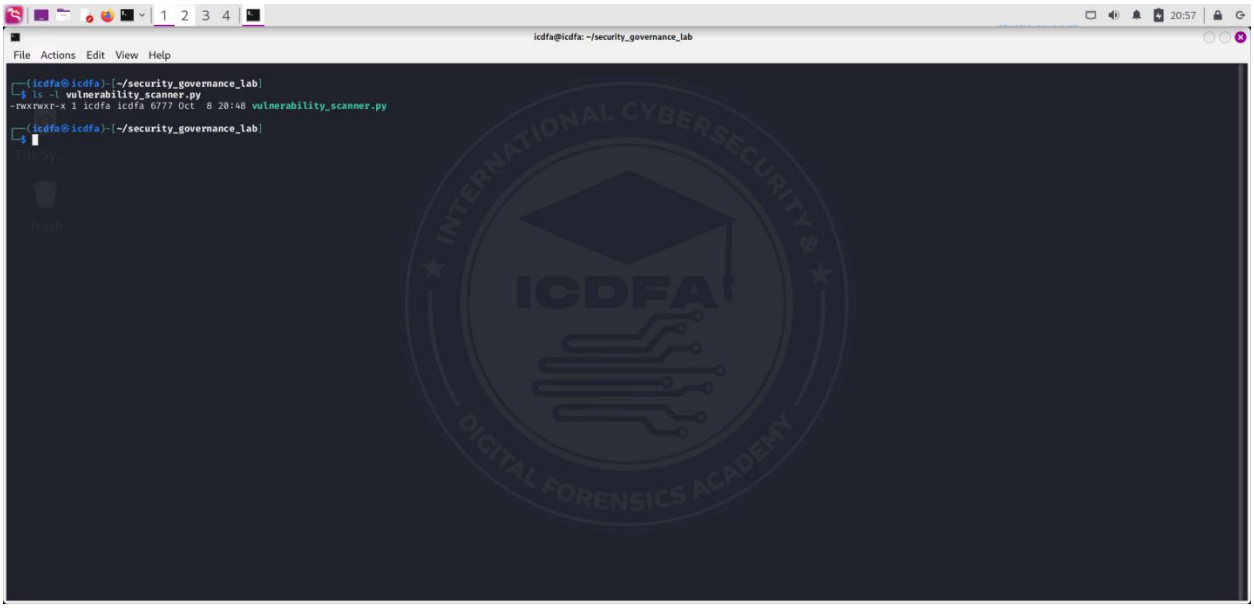


Figure 1.2: Vulnerability_scanner.py created and made executable

I created vulnerability_scanner.py to check my lab for three security issues: the Apache Struts S2-045 vulnerability, weak MySQL credentials, and lack of network separation between the web server and database.

The script uses a harmless test request to check for the Struts flaw, tests the database login and network access, and saves the findings with their severity in a timestamped JSON report. I also used chmod +x to make the script executable and confirmed the permission using ls –l.

The scanner was using http://web_server:8080, which only works inside Docker. Since I ran it from my host, I changed it to http://localhost:8080 because the container's port 8080 is mapped to the host.

Task 3: Set up a patch management system

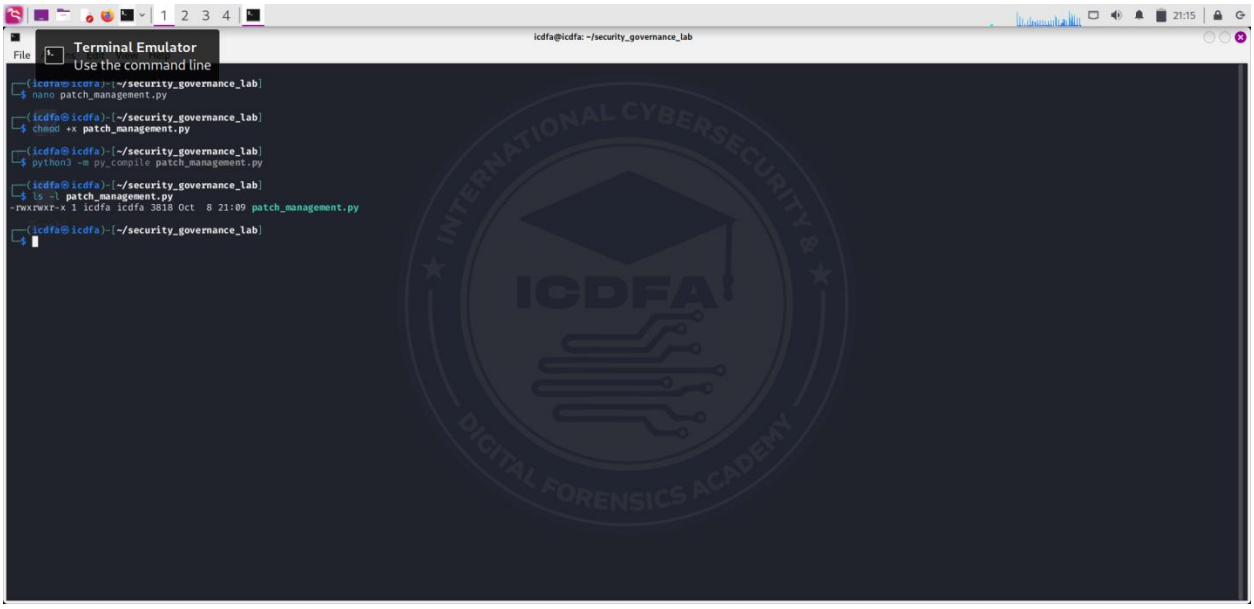


Figure 1.3: Patch_management.py created and made executable

I created patch_management.py to track and apply fixes for the three vulnerabilities in my lab. It includes patches for Apache Struts (Critical), MySQL access control (High), and network segmentation (Medium).

Task 4: Set up a security governance tracking system

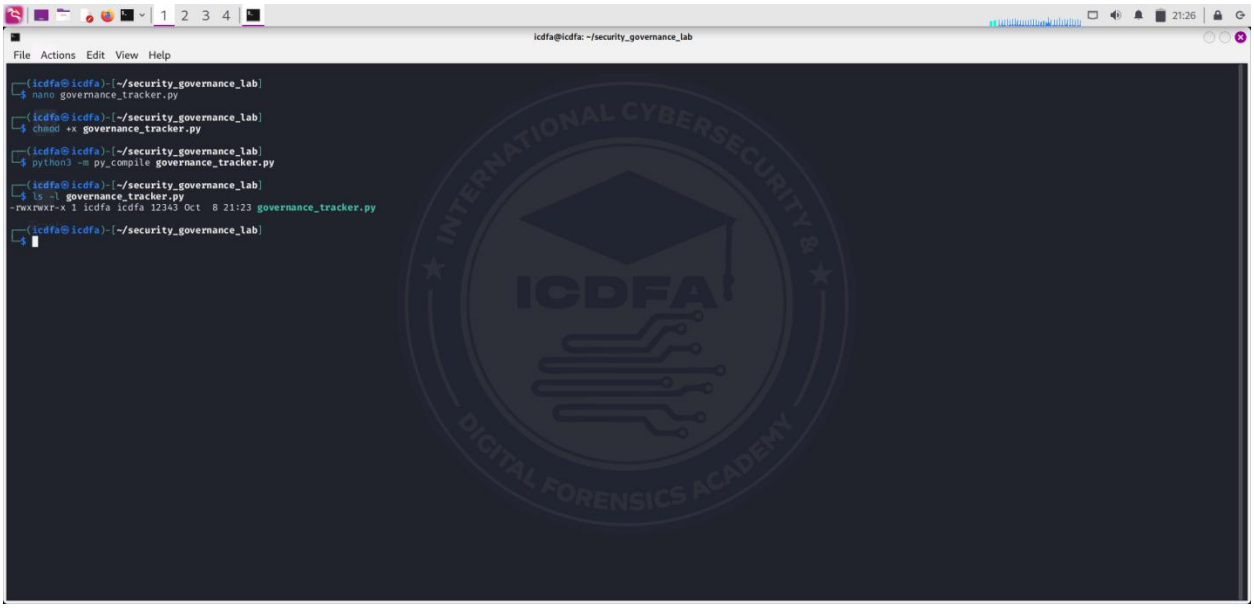


Figure 1.4: governance_tracker.py created and made executable

I created governance_tracker.py to track security policies, controls, risks, incidents, metrics, and audits in one JSON file. It also rates risks based on likelihood and impact and generates a dashboard summary. This relates to the governance gaps in the Equifax breach, where risks and controls were not properly managed.

Task 5: Initialize the governance tracker with sample data

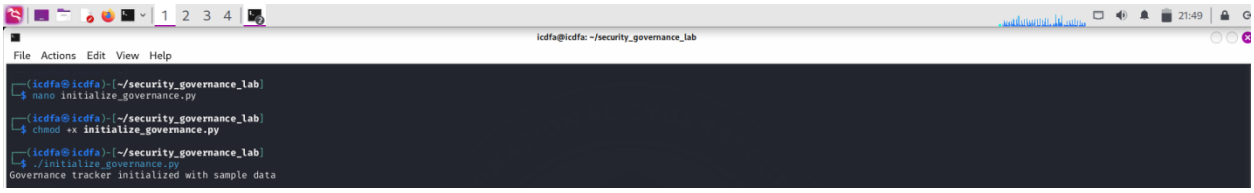


Figure 1.5a: Initialize_governance

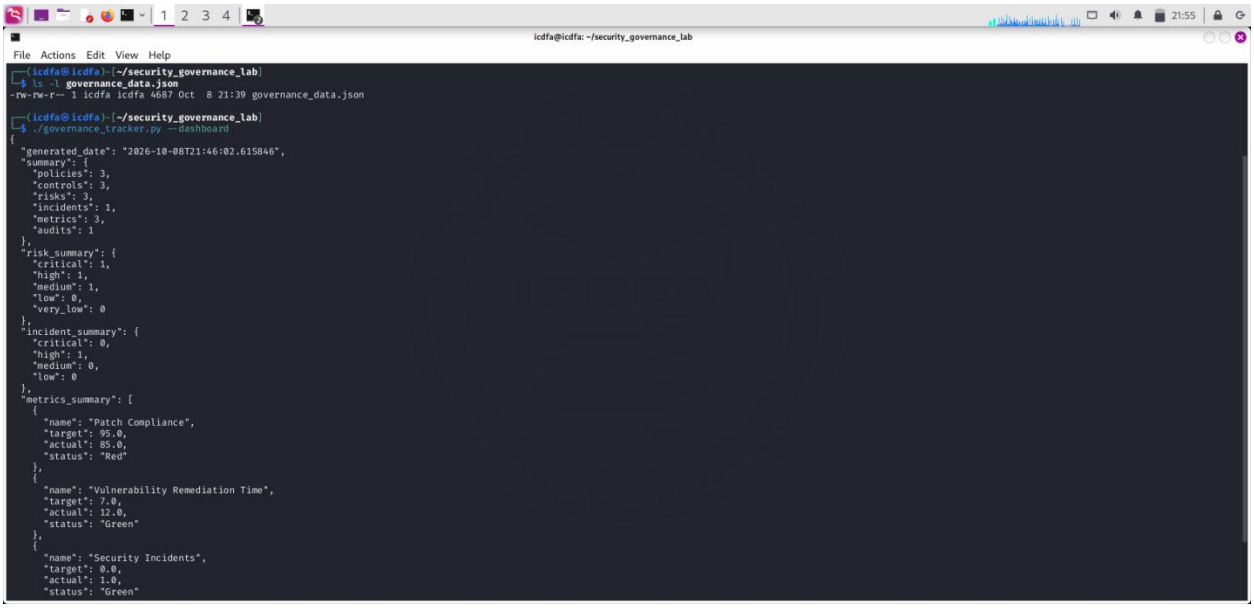


Figure 1.5b: Baseline_dashboard

I ran initialize_governance.py to populate my tracker with sample data. The dashboard shows 3 policies, 3 controls, 3 risks, 1 incident, 3 metrics, and 1 audit. I also noticed that two metrics were marked Green despite missing their targets because the script treats all metrics as if higher values are always better.

Task 6: Run the vulnerability scanner to identify issues

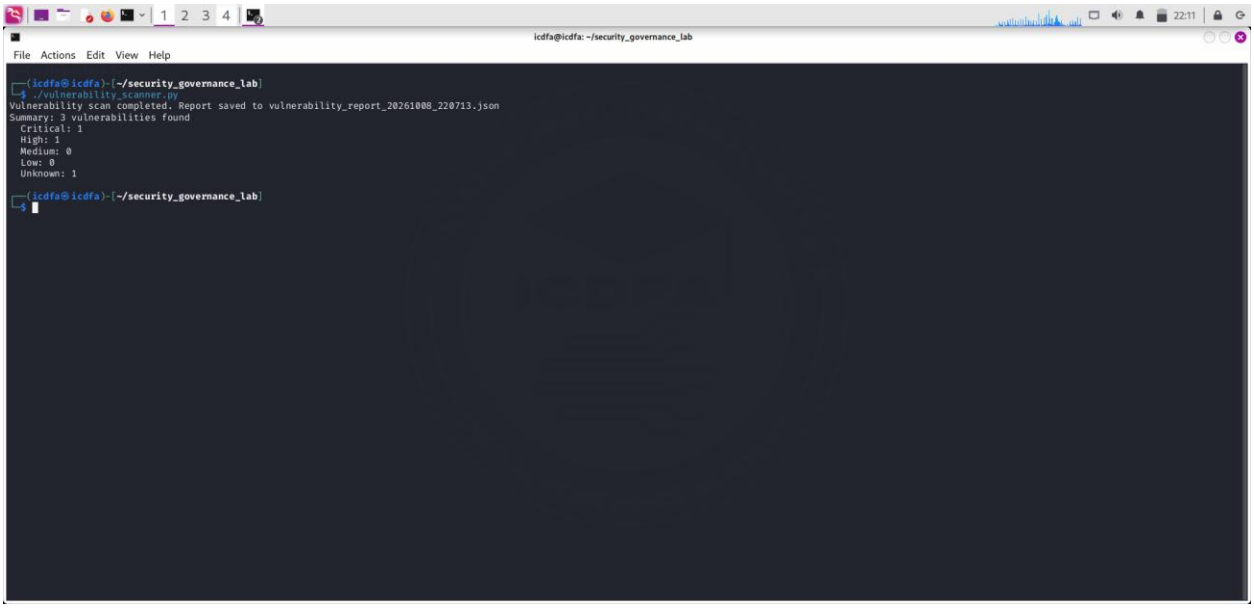


Figure 1.6a: Scanner_run

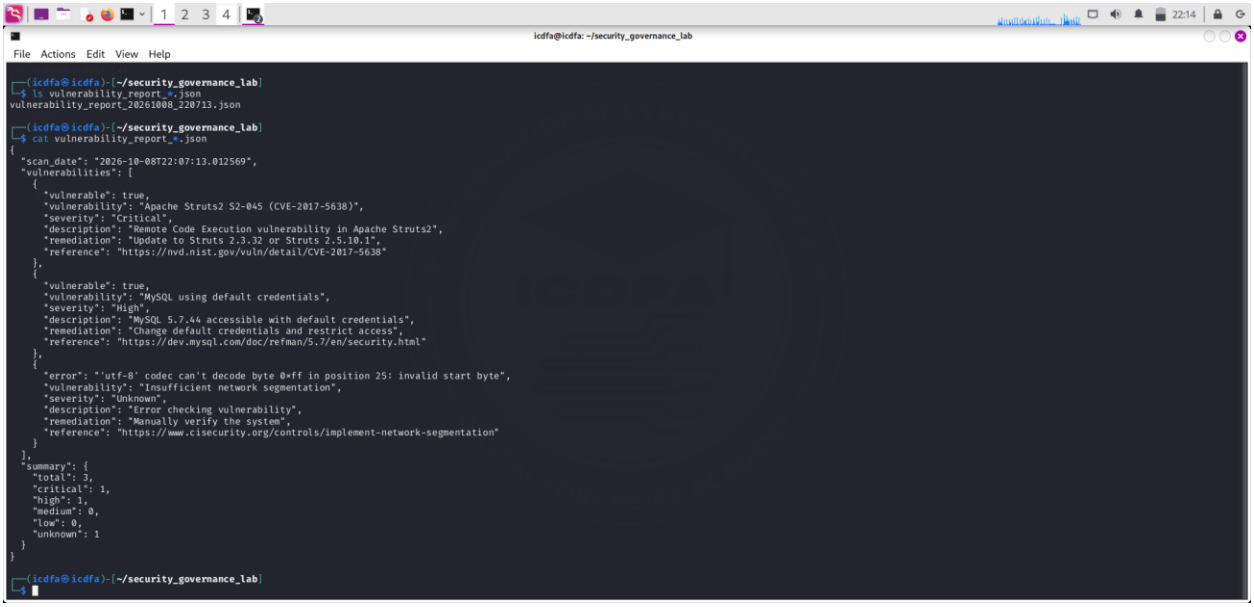


Figure 1.6b: Vulnerability_report

I ran vulnerability_scanner.py and found 3 issues: 1 Critical, 1 High, and 1 Unknown. The web server was vulnerable to Apache Struts S2-045, the same flaw involved in the Equifax breach, while MySQL accepted the default root credentials.

PART 2: Analyzing Security Governance Failures

Task 1: Create a script to simulate an attack on vulnerable systems

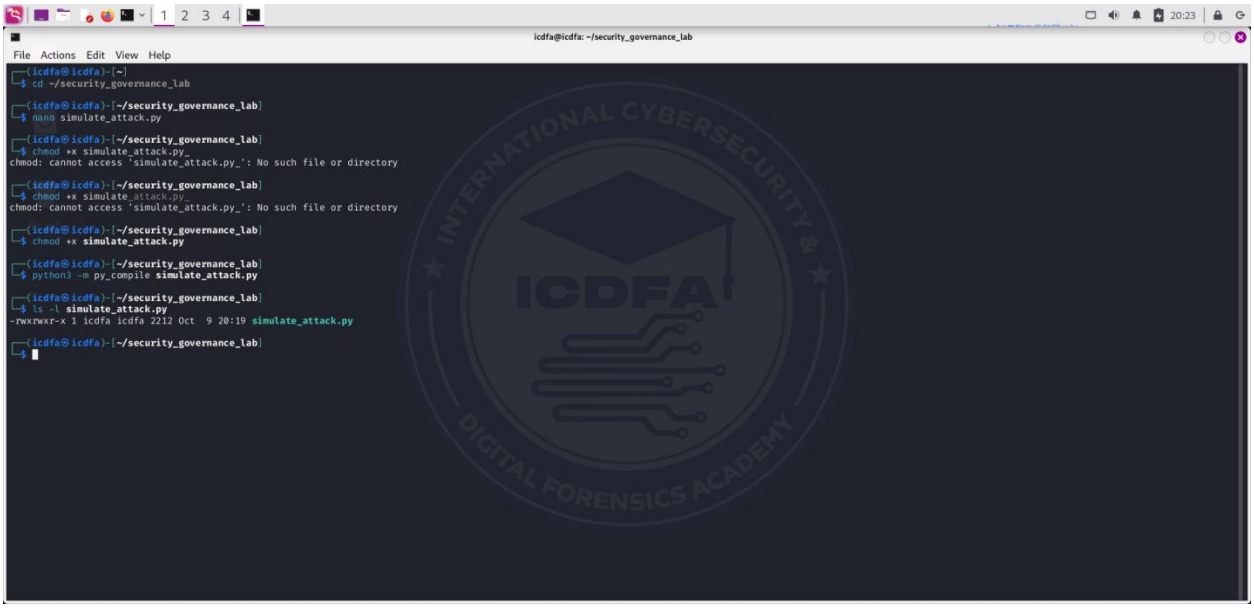


Figure 2.1: Simulate_attack.py created and made executable

I created simulate_attack.py to check if the web server is vulnerable to Struts S2-045 and simulate the stages of a data theft. It records the simulated attack in attack_log.json. No real data is accessed or copied. The log will help me identify governance gaps, similar to what investigators looked into after the Equifax breach.

Task 2: Create a script to analyze security logs and identify governance failures

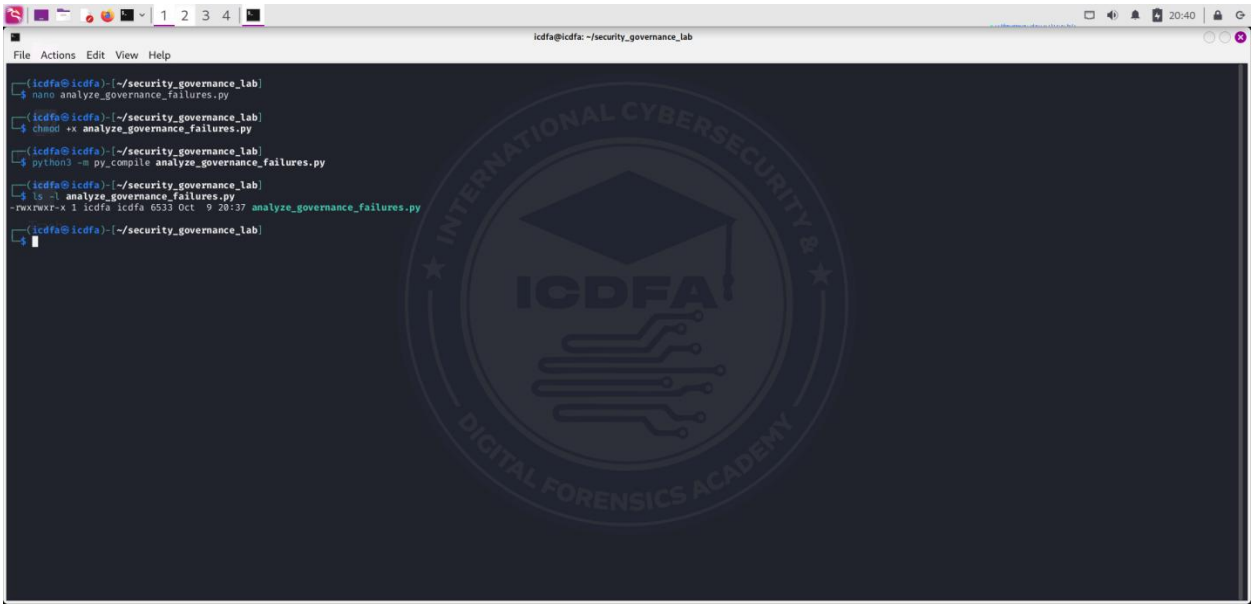


Figure 2.2: analyze_governance_failures.py created and made executable

I created analyze_governance_failures.py to review my scan report, attack log, governance data, and patch status. It identifies governance failures, groups them by category, and saves the results in a timestamped JSON report. This relates to the Equifax breach, where the problem was not just a missing patch but also failures in the processes around it.

Task 3: Run the attack simulation



Figure 2.3a: output of simulate_attack.py confirming the web server is vulnerable to struts S2-045 and simulating database access and data exfiltration

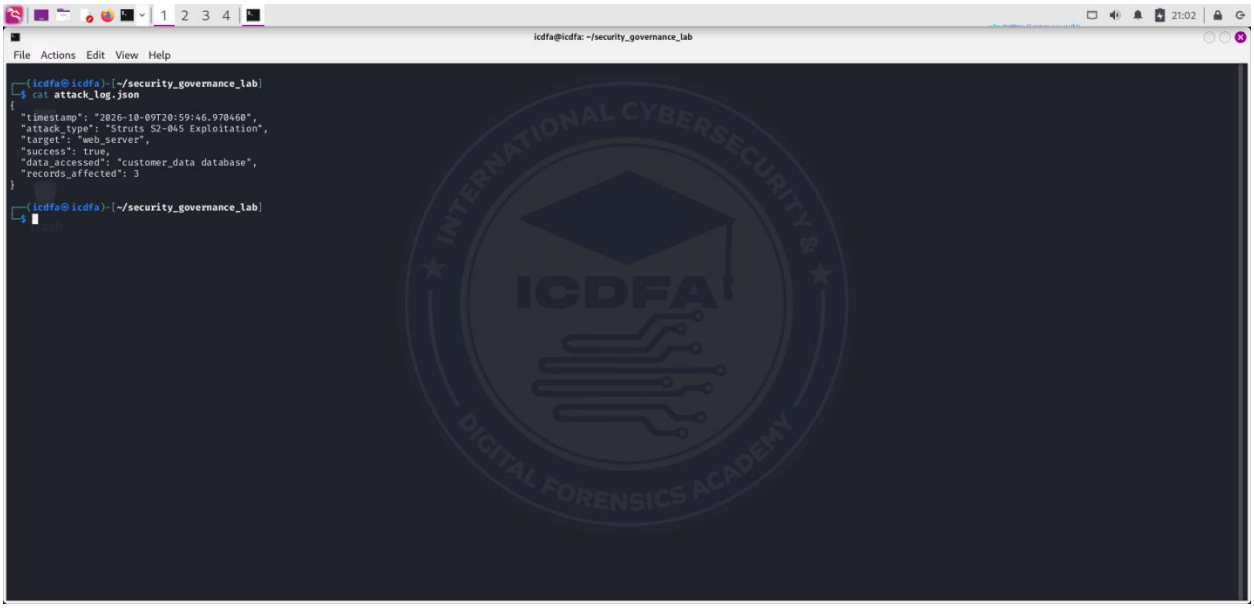


Figure 2.3b: attack_log.json recording the simulate attack against the web server

After restarting the containers, I ran simulate_attack.py (Figure 2.3a). It confirmed the Struts S2-045 vulnerability and simulated database access and data theft. The log (Figure 2.3b) recorded a successful

attack affecting 3 records. The simulation shows how an unpatched web application can expose sensitive data, similar to the Equifax breach.

Task 4: Analyze the governance failures

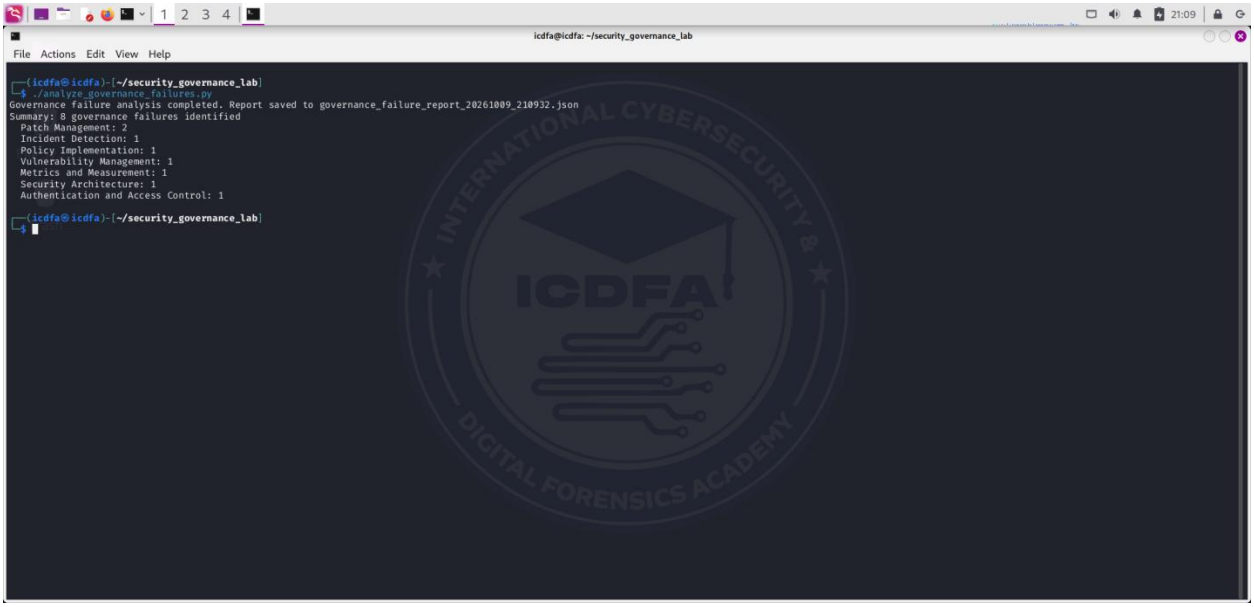


Figure 2.4a: output of analyze_governance_failures.py summarizing the governance failures identified by category

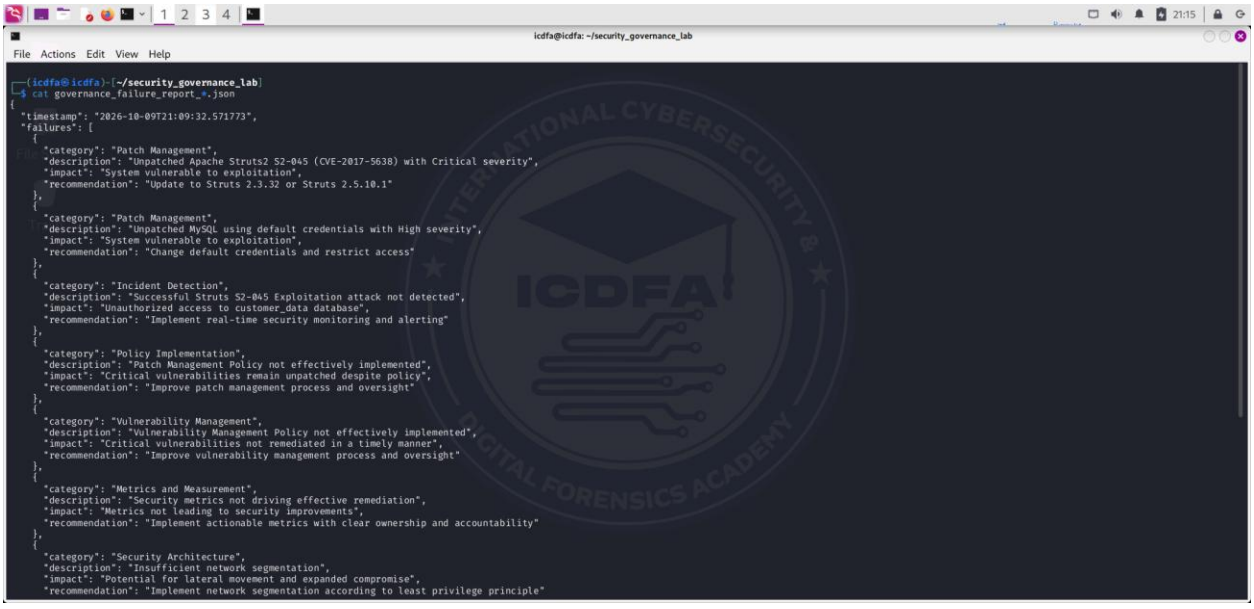


Figure 2.4bi: governance_failure_report_*.json listing each failure with its description, impact and recommendation

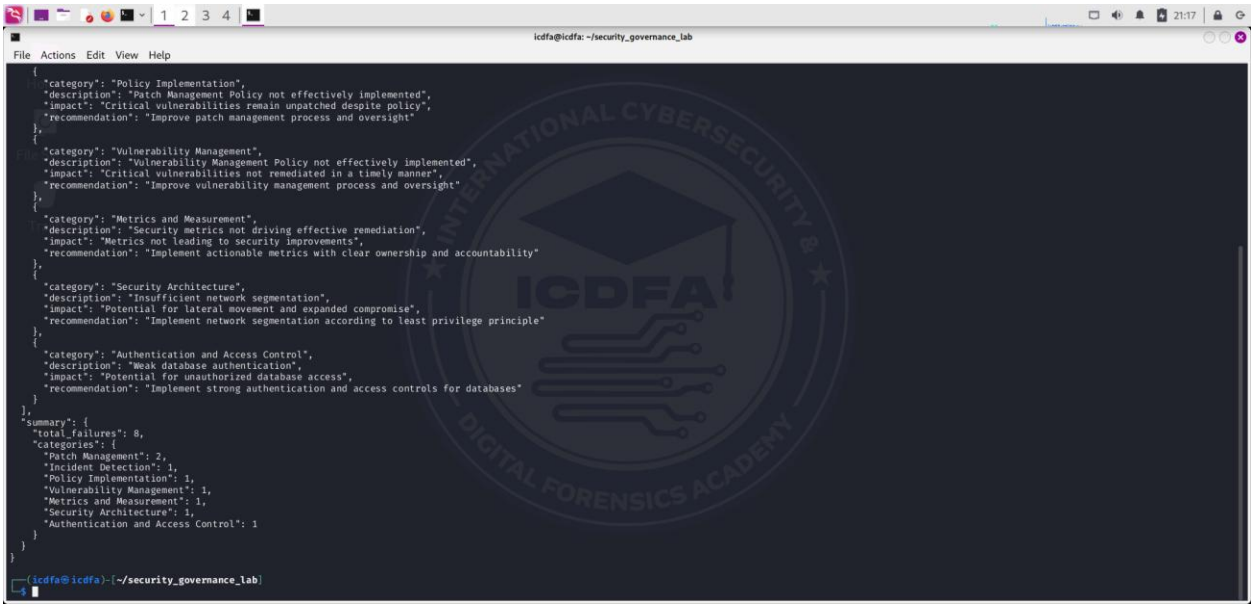


Figure 2.4bii: governance_failure_report_*.json listing each failure with its description, impact and recommendation

I ran analyze_governance_failures.py, which identified 8 governance failures across 7 categories (Figure 2.4a). The report (Figures 2.4bi and 2.4bii) showed that the problem went beyond the unpatched Struts flaw. Policies were not properly implemented, the attack went undetected, metrics did not lead to action, and network and database controls were weak. Like Equifax, the technical flaw was only part of the problem.

Task 5: Create a report comparing the simulated environment to the Equifax case study

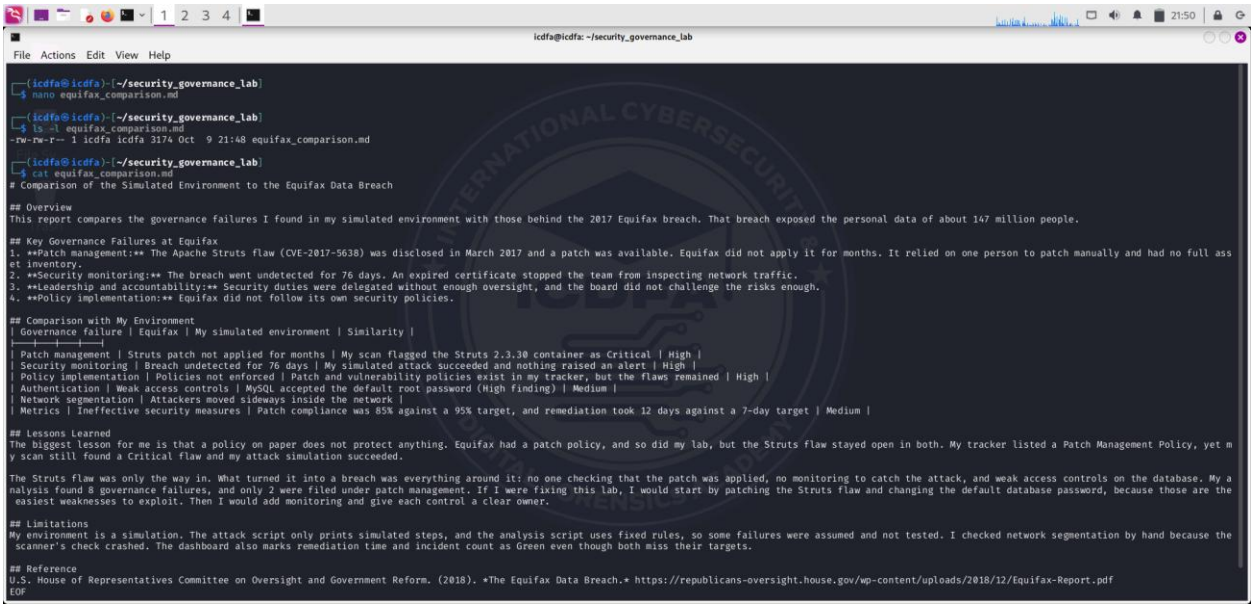


Figure 2.5: Equifax_comparison.md comparing the governance failures in my simulate environment with those in the Equifax breach

PART 3: Implementing Security Governance Controls

Task 1: Create a script to implement security governance controls

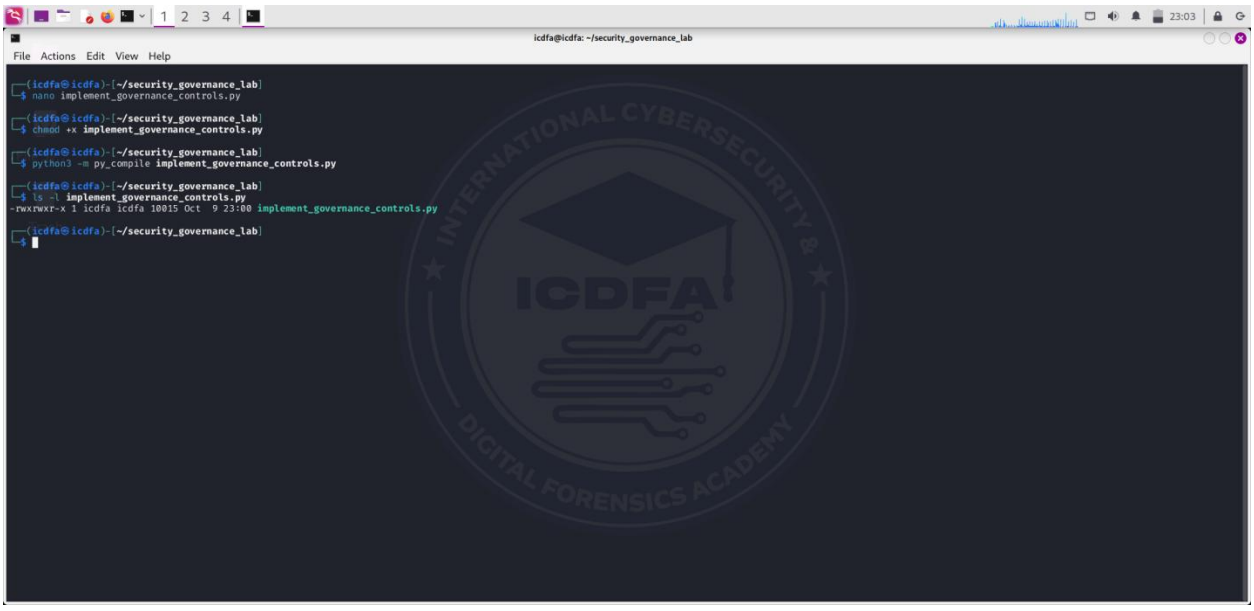


Figure 3.1: Implement_governance_controls.py created and made executable

I created implement_governance_controls.py to address the failures found in Part 2. It covers patch management, network segmentation, database security, monitoring, and governance oversight.

Task 2: Implement the security governance controls



Figure 3.2ai: Output of implement_governance_controls.py –all, applying the patch, network, database, monitoring and oversight controls and then verifying them



Figure 3.2aii: vulnerability scan run automatically after the controls were applied (1 Critical, 1 high, 1 unknown)



Figure 3.2b: patch_status.json after all three patches were applied

I ran implement_governance_controls.py --all (Figure 3.2ai). It added the controls to my tracker, increasing policies from 3 to 4, controls from 3 to 8, and metrics from 3 to 5.

Verification showed that MySQL was secured, but Struts was still vulnerable and the network was not segmented. The patch status file only kept the last patch applied, and the scan still reported 1 Critical, 1 High, and 1 Unknown finding (Figures 3.2aii and 3.2b). This is because the lab patches are simulated and do not actually change the running containers.

Task 3: Create a report documenting the implemented controls

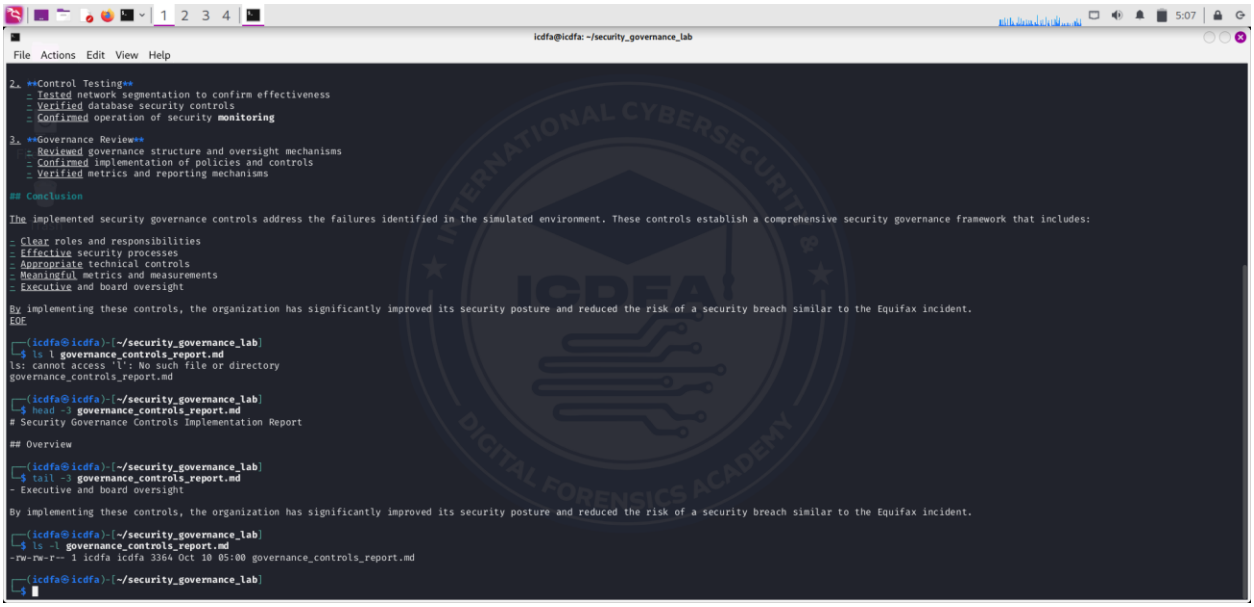


Figure 3.3: governance_controls_report.md documenting the controls implemented for each governance failure

The controls report describes the intended end state. My verification evidence (Figures 3.2a to 3.2b) shows the controls were recorded in the tracker and the patch status, but the post-control scan still reported the Critical and High findings, because the patches in this lab are simulated.

Task 4: Verify the effectiveness of the implemented controls



Figure 3.4a: output of implement_governance_controls.py –verify

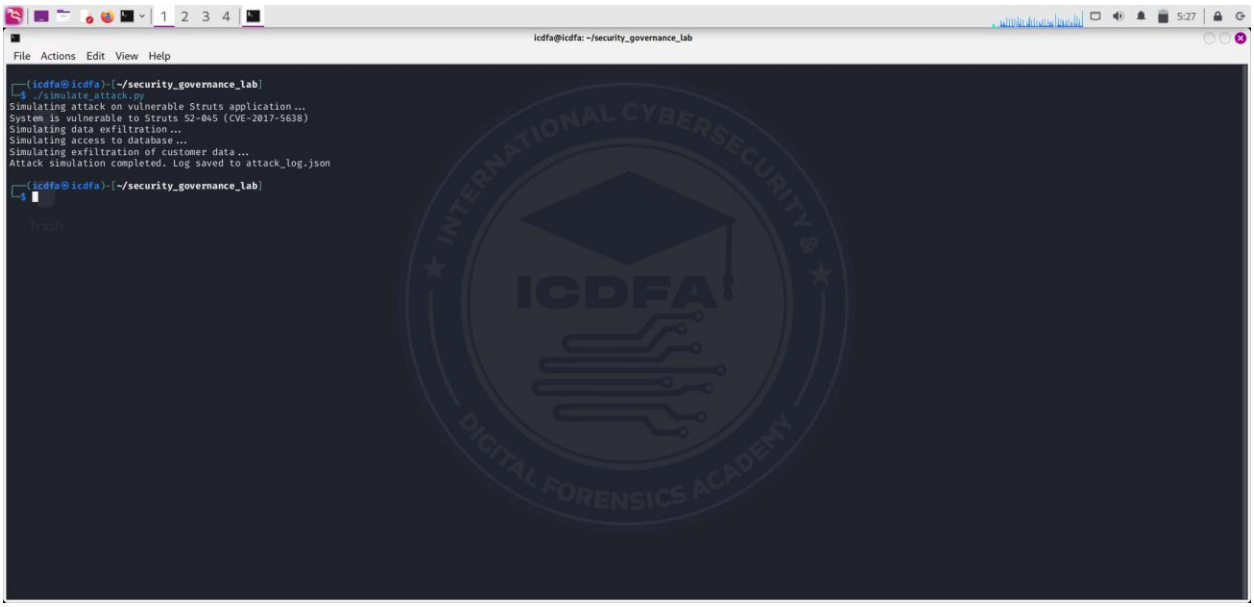


Figure 3.4b: output of simulate_attack.py run again after the controls were applied

I ran implement_governance_controls.py --verify (Figure 3.4a). My tracker showed 4 policies, 8 controls, and 5 metrics. MySQL was secured, but Struts was still vulnerable and the network was not segmented. The new scan still found 3 issues, and simulate_attack.py confirmed that the simulated attack could still succeed (Figure 3.4b).

This showed that the controls were recorded in the tracker but did not change the running containers. In a real environment, I would patch Struts, change the MySQL password, restrict the webserver's access to the database, and run another scan to confirm the fixes.

How the implemented controls address the governance failures identified in the Equifax case study

Equifax had failures in patching, monitoring, accountability, and policy implementation. The controls I added target these issues. Patch management addresses the missed Struts patch, monitoring targets delayed breach detection, and network segmentation and database security limit attackers' access. The governance policy also defines the roles of the CEO, CISO, and board.

However, my lab showed that these controls were only recorded, not fully working. Struts was still vulnerable, and the simulated attack still succeeded. This taught me that having controls in place on paper is not enough; they must work and be tested.

PART 4: Developing Security Governance Metrics and Reporting

Task 1: Create a script to generate security governance metrics

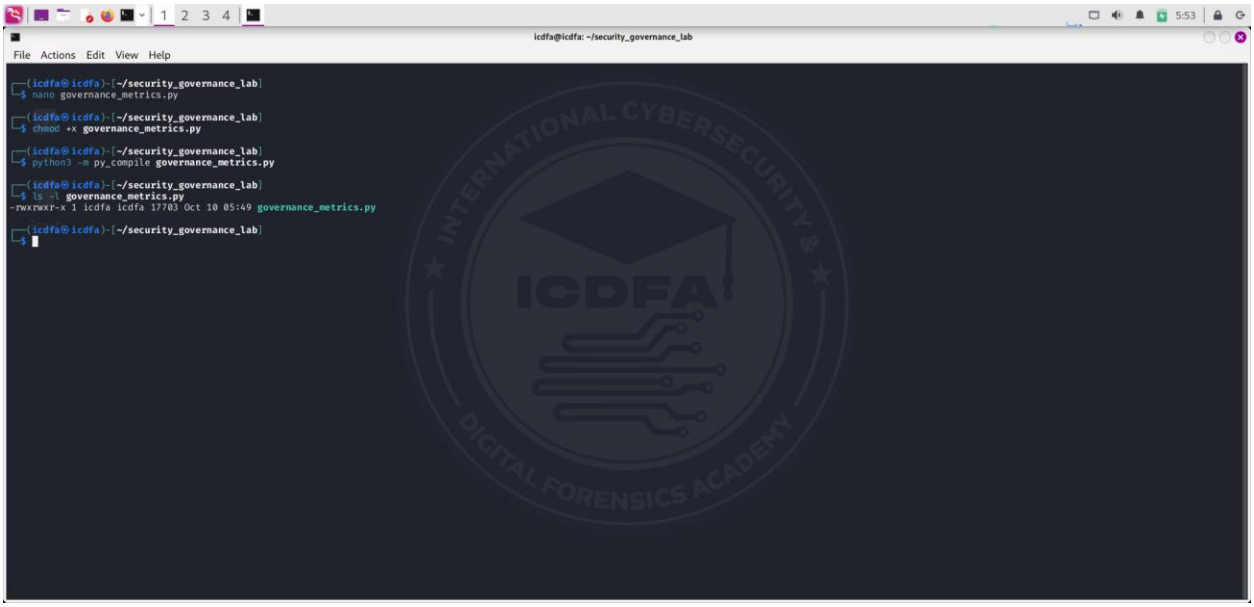


Figure 4.1: governance_metrics.py created and made executable

I created governance_metrics.py to generate an executive dashboard, a board report, and five charts showing patch compliance, vulnerability remediation time, security incidents, monitoring coverage, and policy compliance.

Task 2: Generate security governance metrics and reports

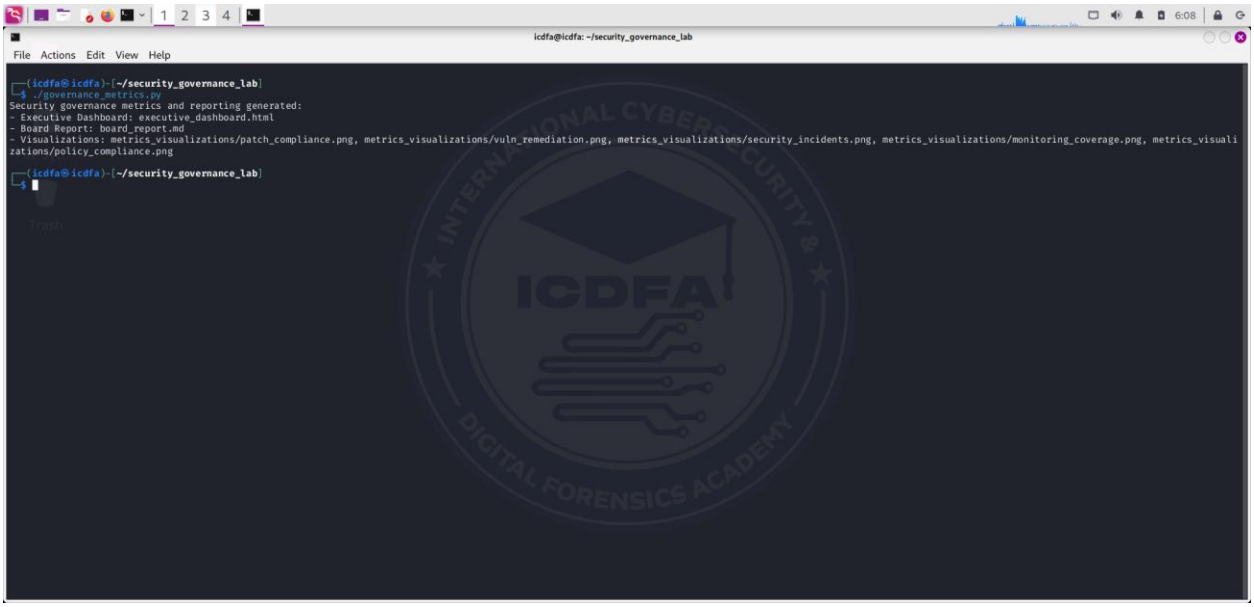


Figure 4.2a: Output of governance_metrics.py generating the executive dashboard, board report and five trends chart

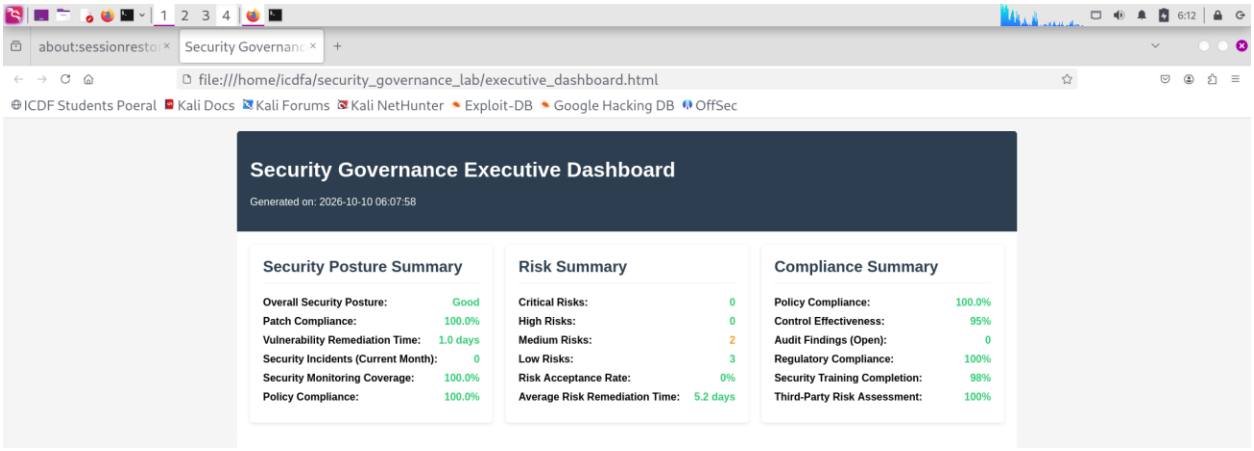


Figure 4.2bi: Executive dashboard showing the security posture, risk and compliance summary card

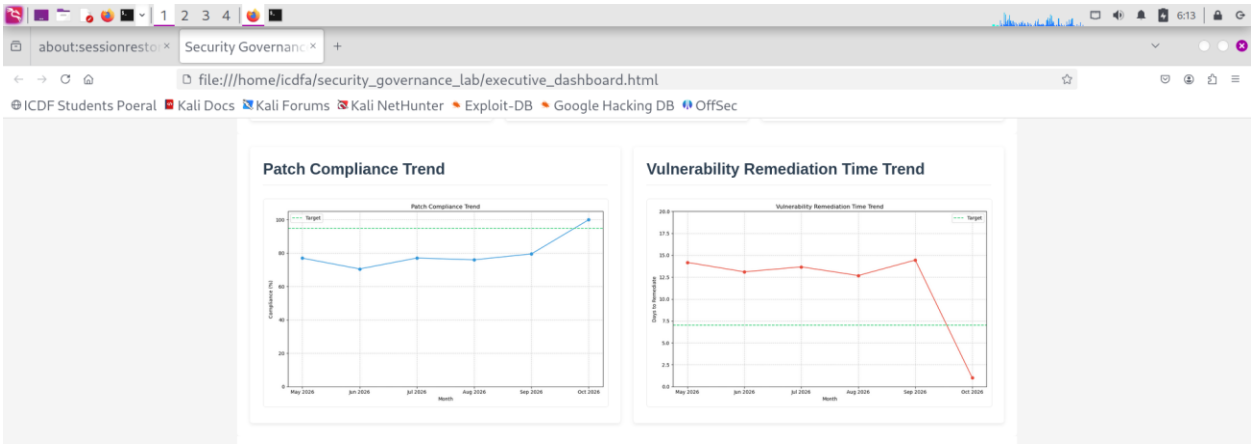


Figure 4.2bii: Dashboard trend charts for patch compliance and vulnerability remediation time.

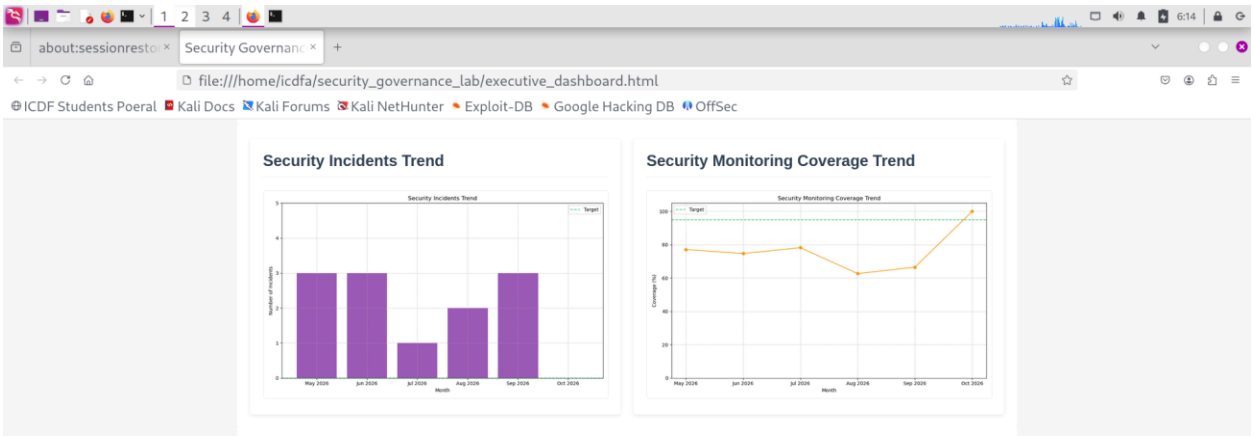


Figure 4.2biii: Dashboard trend charts for security incidents and security monitoring coverage

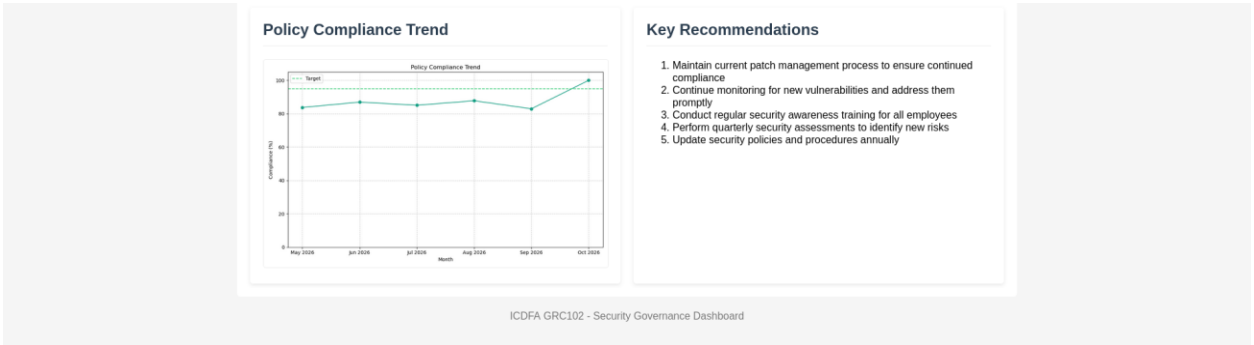


Figure 4.2biv: Dashboard policy compliance trend and key recommendations

I generated the dashboard (Figures 4.2a–4.2biv), board report, and five trend charts. The dashboard showed a Good security status with 0 critical risks.

Task 3: Create a document explaining the security governance metrics framework

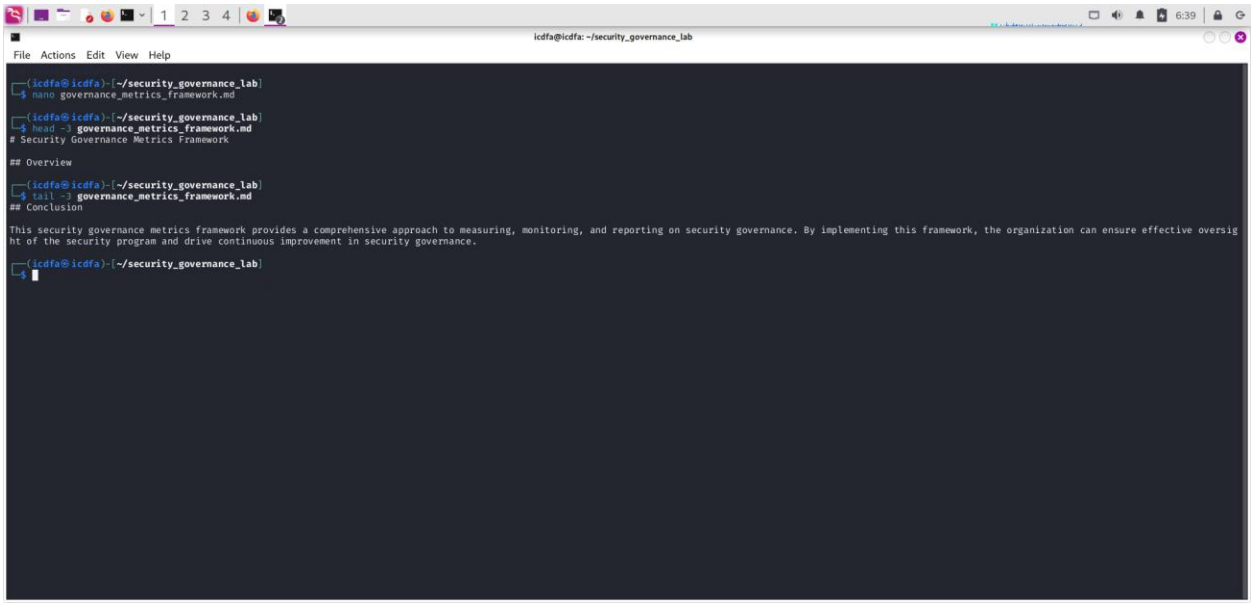


Figure 4.3: governance_metrics_framework.md defining the metric categories, reporting mechanisms and lifecycle

Task 4: Create a security governance dashboard user guide

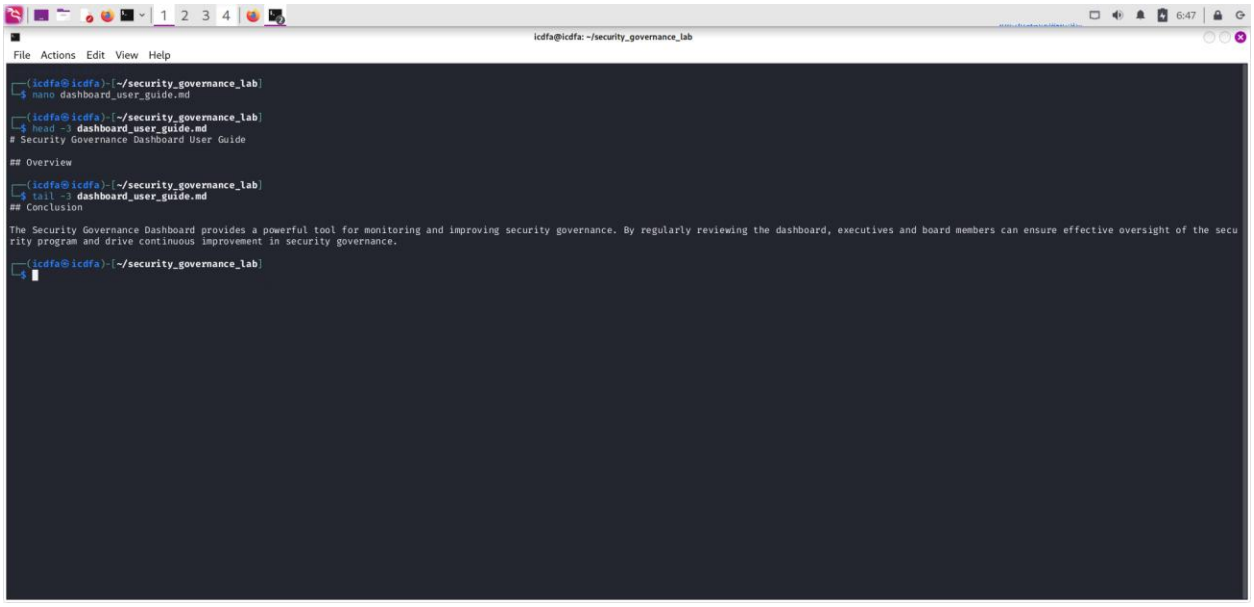


Figure 4.4: Dashboard_user_guide.md explaining how to open, read and refresh the executive dashboard

I created dashboard_user_guide.md which explains how executives and board members can read the dashboard, understand the colour indicators, interpret the trend charts, and refresh the dashboard.

A brief explanation of how the metrics and reporting mechanisms support effective security governance

Metrics and reporting help executives and the board know if security controls are actually working. A policy can exist on paper while the problem remains, as seen in the Equifax breach. The dashboard brings key metrics like patch compliance, remediation time, incidents, and monitoring coverage together, while the board report helps non-technical leaders understand the results and take action.

LINK TO THE GITHUB

https://github.com/EgbagbeEsther/ICDFA-Assignment/tree/main/GRC102_Lab_Assignment/GRC102_W5_Lab

NOTE: For most of the scripts I created, I ran **python3 -m py_compile** to check for syntax errors and **ls -l** to confirm the file existed and had the right permissions. This helped me catch typing mistakes early and confirm that the files were created correctly.